

• TREINAMENTO TÉCNICO

Versionamento de Código e GitHub

Fundamentos de Git, colaboração e automação para equipes

 55 minutos

 Prática ao vivo

 Nível introdutório

● OBJETIVO

O que você vai levar desta aula

- ◆ Entender **por que versionar** código evita retrabalho e perda de histórico.
- ◆ Distinguir claramente **Git** (a ferramenta) de **GitHub** (a plataforma).
- ◆ Executar o fluxo essencial: **clonar** → **branch** → **commit** → **push** → **Pull Request** → **merge**.
- ◆ Conhecer as principais ferramentas do GitHub para colaboração e automação.

“Quem já precisou recuperar uma versão antiga de um arquivo — ou trabalhou com um `projeto_final_v3_corrigido_agora.zip`?”

Sem versionamento, o time convive com:

- ◆ Perda de alterações
- ◆ Falta de rastreabilidade
- ◆ Arquivos duplicados
- ◆ Conflitos entre pessoas

Não são a mesma coisa

Git

A ferramenta

Controle de versão que roda **localmente**, no seu computador. É o mecanismo que registra a evolução do projeto.

GitHub

A plataforma

Serviço **online** que hospeda repositórios Git e acrescenta colaboração, revisão, organização e automação.

● 3 · CONCEITOS FUNDAMENTAIS — 10 MIN

Repositório	Projeto e todo o seu histórico	Push	Envio de alterações para o GitHub
Commit	Registro de um conjunto de alterações	Pull	Atualização do projeto local
Branch	Linha de trabalho independente	Pull Request	Pedido de revisão e incorporação
Merge	União de alterações	Conflito	Alterações incompatíveis a resolver
Clone	Cópia do repositório para o computador		

● 3 · O FLUXO EM UMA LINHA

Alterar arquivos → `git add` → `git commit` → `git push`
→ `Pull Request` → revisão → `merge`

Passo 1 · Clonar o projeto

```
$ git clone URL_DO_REPOSITORIO  
$ cd nome-do-projeto
```

Clonar traz o repositório inteiro — código e histórico — para a sua máquina.

Passo 2 · Criar uma branch

```
$ git switch -c melhoria-readme  
# alternativa: git checkout -b melhoria-readme
```

A branch é uma **linha de trabalho isolada** — você altera sem tocar na versão principal do projeto.

Passo 3 · Alterar, verificar e registrar

```
# edite o README.md, depois:  
$ git status  
$ git add README.md  
$ git commit -m "Atualiza documentação do projeto"
```

Passo 4 · Enviar para o GitHub

```
$ git push origin melhoria-readme
```

O **push** publica a sua branch no GitHub, deixando-a disponível para o time revisar.

Passo 5 · Criar o Pull Request

- Selecione a branch enviada
- ↺ Crie o Pull Request
- ↻ Título e descrição objetivos
- 🔍 Solicite uma revisão
- 5 Comentários → aprovação → merge

Para que serve o PR

- ◆ Revisar código
- ◆ Discutir soluções
- ◆ Identificar problemas
- ◆ Registrar decisões
- ◆ Rodar validações automáticas

Comandos que vale a pena conhecer — próximos passos

git init

Começa a versionar uma pasta do zero

git log --oneline

Mostra o histórico de commits, resumido

git diff

Mostra exatamente o que mudou nos arquivos

git merge <branch>

Junta o trabalho de uma branch na outra

git pull

Traz as novidades do GitHub para o projeto local

git branch

Lista as branches existentes

git restore <arquivo>

Descarta alterações ainda não salvas

git remote -v

Mostra a qual repositório remoto você está ligado



Repositories

Guardam projeto, documentação e histórico. Mostre README, commits, branches, tags, releases e permissões.



Pull Requests

Comentários, aprovação, solicitação de mudanças e comparação entre branches.

● 5 · FERRAMENTAS DO GITHUB

Issues

Registram erros, tarefas, melhorias, dúvidas e demandas técnicas.

Corrigir erro ao exportar relatório quando o campo de data está vazio.

Projects

Organizam o trabalho num quadro visual:

Backlog → Em andamento → Em revisão → **Concluído**

● 5 · FERRAMENTAS DO GITHUB

GitHub Actions

Automatizam testes, validações, builds, publicação de docs e deploy.

Alteração enviada → **Action** valida →
resultado no Pull Request

Releases e Tags

Marcam versões importantes do projeto.

v1.0.0 → v1.1.0 → **v2.0.0**

● 6 · BOAS PRÁTICAS E SEGURANÇA — 5 MIN

- ◆ Commits pequenos e objetivos
- ◆ Mensagens claras
- ◆ Trabalhar em branches específicas
- ◆ Evitar mexer direto na branch principal
- ◆ Revisar antes do merge
- ◆ Manter o README atualizado
- ◆ Usar Issues para organizar demandas
- ◆ Manter um `.gitignore` adequado
- ◆ Atualizar o projeto local antes de começar
- ◆ **Nunca** versionar senhas, tokens ou dados sensíveis

● 6 · ESCREVA BOAS MENSAGENS

EVITE

alterações

ajustes

correção

PREFIRA

Corrige validação de CPF no cadastro de clientes

Atualiza instruções de instalação do projeto

Adiciona filtro por período no relatório de vendas

● 7 · PERGUNTAS E ATIVIDADE — 3 MIN

- ◆ O que aconteceria sem versionamento?
- ◆ Qual a diferença entre Git e GitHub?
- ◆ Para que serve uma branch?
- ◆ Qual a finalidade de um Pull Request?
- ◆ Onde registrar uma nova tarefa ou problema?
- ◆ O que pode ser automatizado por Actions?

Com computadores: cada participante cria uma branch, altera o README e abre um Pull Request.

Issue → **Branch** → Alteração → **Commit** → **Push**
→ **Pull Request** → Revisão → **Merge**

● MENSAGEM PRINCIPAL

Versionamento não é apenas guardar cópias do código.
É controlar mudanças, colaborar com segurança e manter o histórico das decisões do projeto.